

THE UNIVERSITY OF MELBOURNE

COMP90038 Algorithms and Complexity

Lecture 4: Analysis of Algorithms

Olya Ohrimenko
(with thanks to Harald Søndergaard)

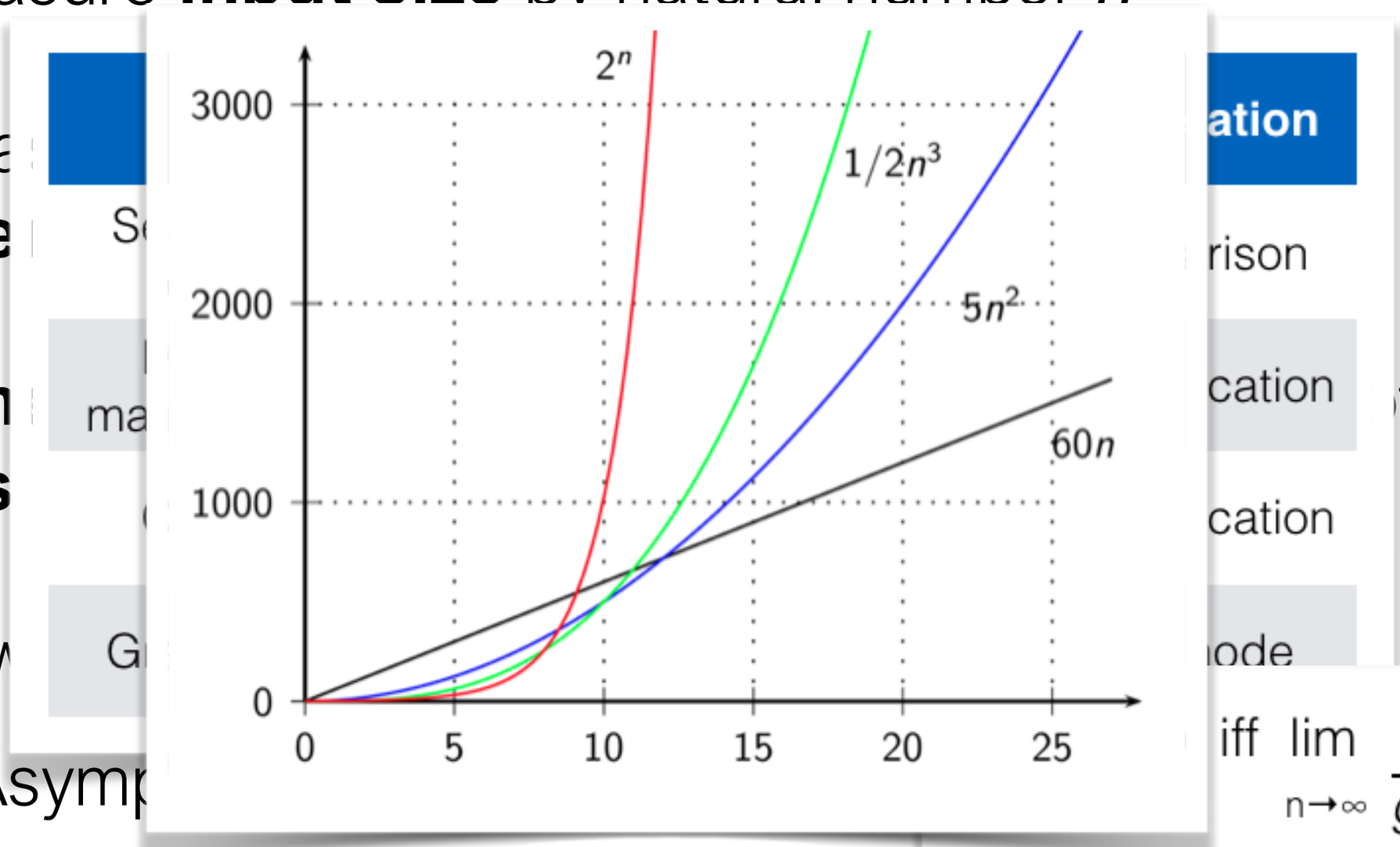
Announcements

Meet student representatives (see Welcome Module on Canvas for details)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Last Time: Time Complexity

- Measure **input size** by natural number *n*
- Mea
  ope
- **Tim**
  bas
- How
  - Asymp
  - $O(g(n))$, $\Omega(g(n))$, $\Theta(g(n))$



ation

rison

cation

f

cation

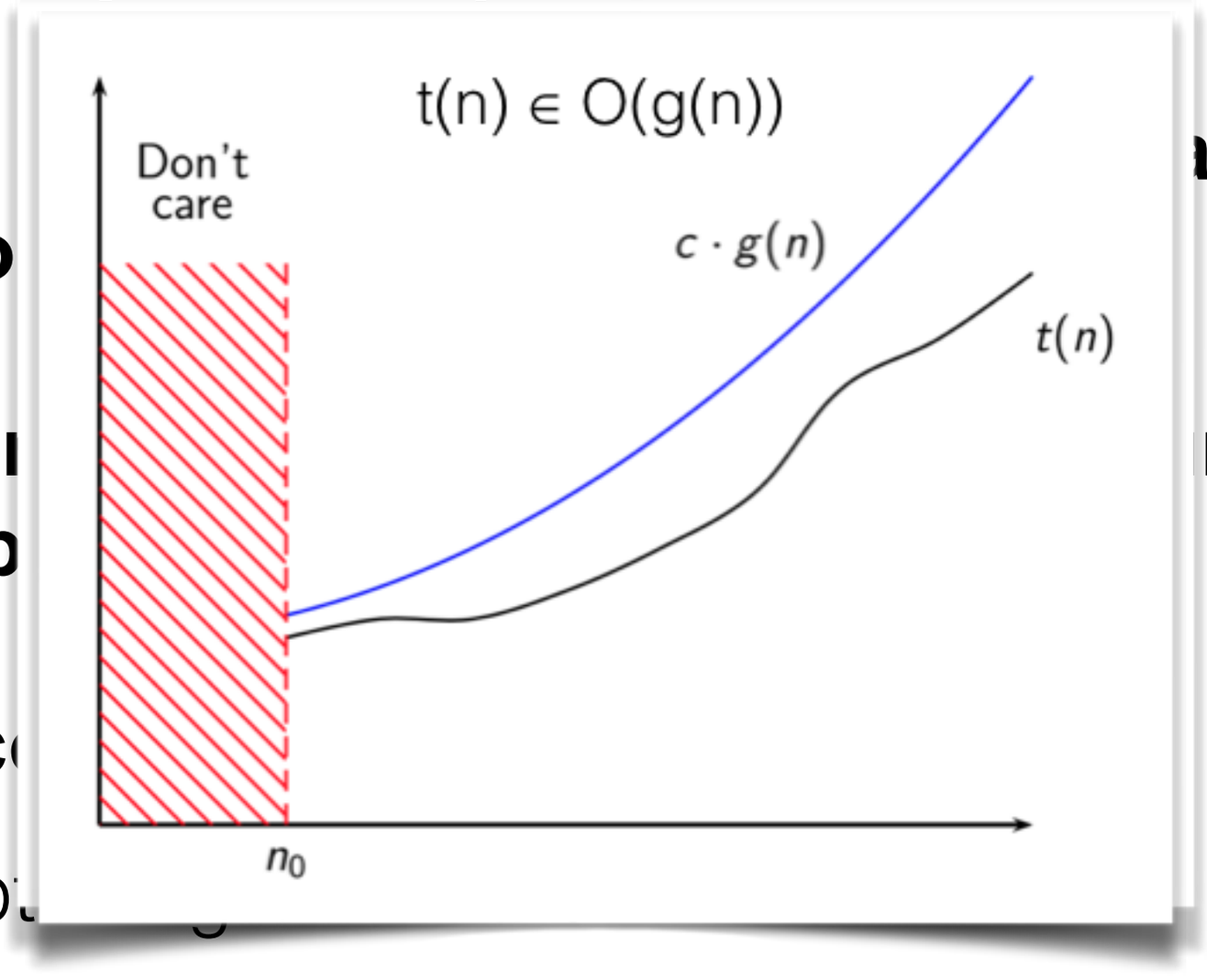
ode

iff $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Last Time: Time Complexity

- Measure **input size** by natural number *n*
- Measure **operatio** **asic**
- **Time co** mber of **basic op**
- How to **c**
  - Asymp
  - *O*(g(n)), Ω(*g*(*n*)), Θ(*g*(*n*))



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Establishing Growth Rate

- We proved $t(n) \in O(g(n))$ for some cases of t and g, using the definition of O directly:

  $n > n_0 \Rightarrow t(n) < c \cdot g(n)$ for some c and n_0.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Establishing Growth Rate

- We proved $t(n) \in O(g(n))$ for some cases of t and g, using the definition of O directly:

  $n > n_0 \Rightarrow t(n) < c \cdot g(n)$ for some c and n_0.

- A more common approach uses

- $\lim_{n \to \infty} \frac{f(n)}{g(n)} =$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Establishing Growth Rate

- We proved $t(n) \in O(g(n))$ for some cases of t and g, using the definition of O directly:

  $n > n_0 \Rightarrow t(n) < c \cdot g(n)$ for some c and n_0.

- A more common approach uses

- $$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & f \text{ grows asymptotically slower than } g \\ c & f \text{ and } g \text{ have same order of growth} \\ \infty & f \text{ grows asymptotically faster than } g \end{cases}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

O, Ω, Θ

What this tells us about how $f(n)$ relates to $O(g(n))$, $\Omega(g(n))$ and $\Theta(g(n))$

- $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & f \text{ grows asymptotically slower than } g \\ c & f \text{ and } g \text{ have same order of growth} \\ \infty & f \text{ grows asymptotically faster than } g \end{cases}$

$$f(n) \in \Theta(g(n))$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Show that $1000n \in O(n^2)$

$$\lim_{n \to \infty} \frac{1000n}{n^2}$$

$$= \lim_{n \to \infty} \frac{1000}{n}$$

$$= 0$$

So $1000n$ grows asymptotically slower than n^2

Thus $1000n \in O(n^2)$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

L'Hôpital's Rule

$$\lim_{h\to\infty} \frac{t(n)}{g(n)} = \lim_{h\to\infty} \frac{t'(n)}{g'(n)}$$

where t' and g' are the derivatives of t and g

- For example, show that $\log_2 n$ grows slower than $\sqrt{n}$

$$\lim_{n\to\infty} \frac{\log_2 n}{\sqrt{n}} = \lim_{n\to\infty} \frac{\frac{1}{(\log_e 2)n}}{\frac{1}{2\sqrt{n}}} = \lim_{n\to\infty} \left(\frac{1}{\log_e 2} \cdot \frac{1}{n} \cdot 2\sqrt{n} \right)$$

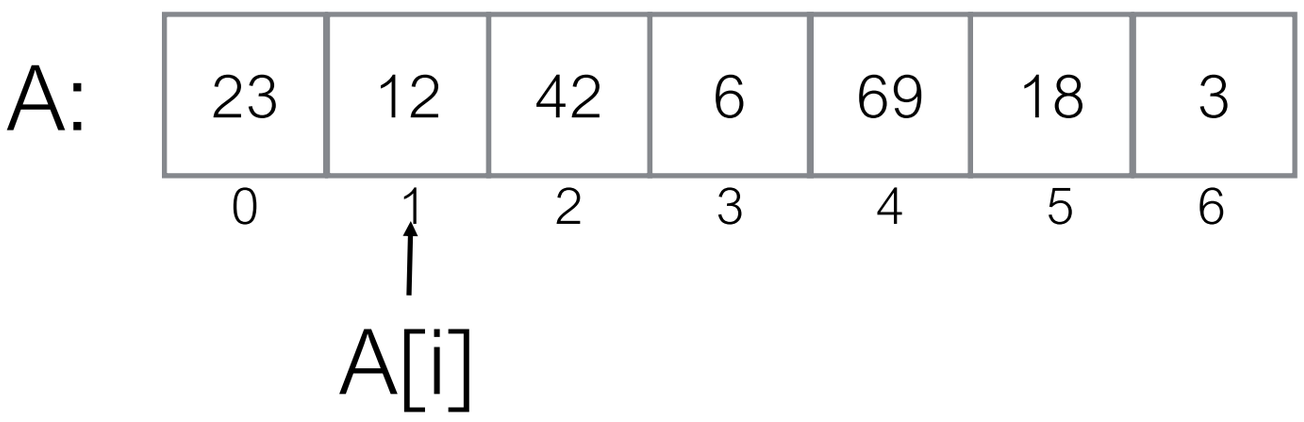
$$= \frac{2}{\log_e 2} \lim_{n\to\infty} \frac{\sqrt{n}}{n} = \frac{2}{\log_e 2} \lim_{n\to\infty} \frac{1}{\sqrt{n}} = 0$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Finding Largest Element in an Array

```
function MaxElement(A[·], n)
    max ← A[0]
    for i ← 1 to n − 1 do
        if A[i] > max then
            max ← A[i]
    return max
```

(where *n* is length of the array)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Finding Largest Element in an Array

```
function MaxElement(A[·], n)
    max ← A[0]
    for i ← 1 to n − 1 do
        if A[i] > max then
            max ← A[i]
    return max
```

(where *n* is length of the array)

Size of input, *n*:
length of the array

Basic operation: comparison “A[i] > max”

Count the number of basic operations executed for an array of size *n:*

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Finding Largest Element in an Array

```
function MaxElement(A[·], n)
    max ← A[0]
    for i ← 1 to n − 1 do
        if A[i] > max then
            max ← A[i]
    return max
```

(where *n* is length of the array)

Size of input, *n*:
length of the array

Basic operation: c

Count the number of basic
for an array of size *n:*

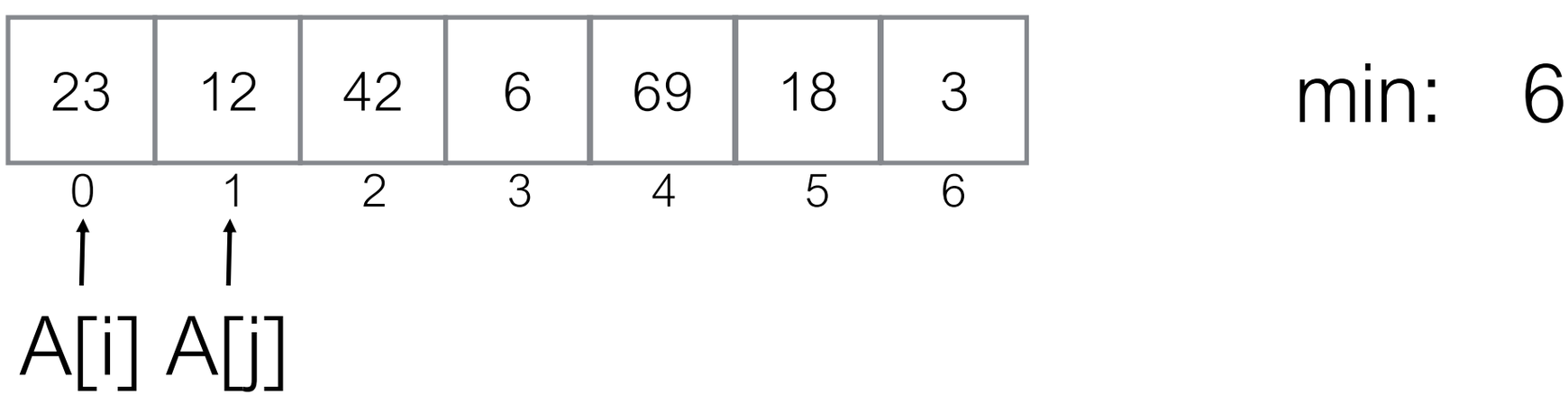
$$\sum_{i=l}^{u} 1 = \underbrace{1+1+\cdots+1}_{u-l+1 \text{ times}} = u-l+1$$

$$C(n) = \sum_{i=1}^{n-1} 1 = ((n-1)-1+1) = n-1 \in \Theta(n)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Selection Sort

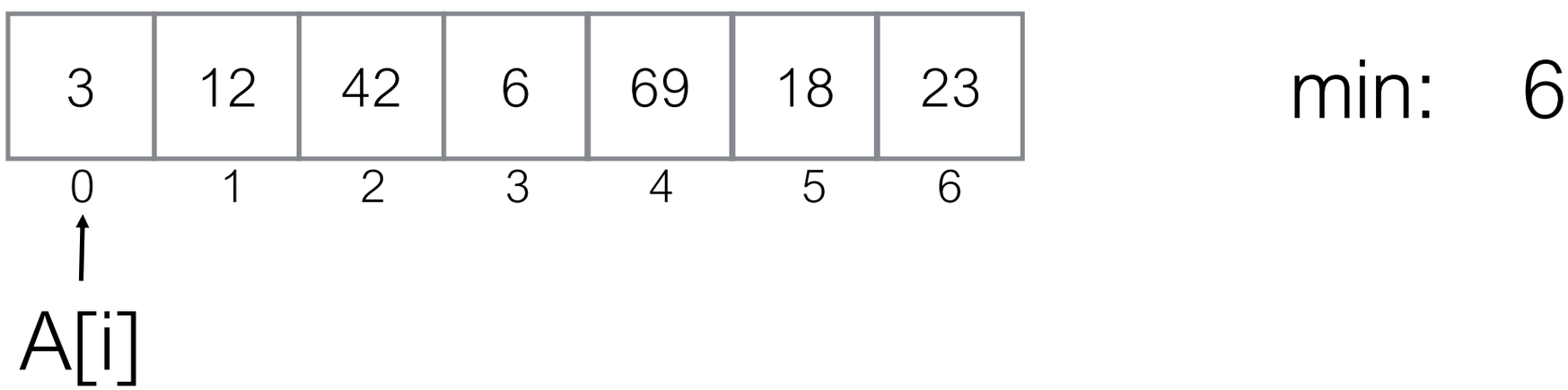
```
function SelSort(A[·], n)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
        swap A[i] and A[min]
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Selection Sort

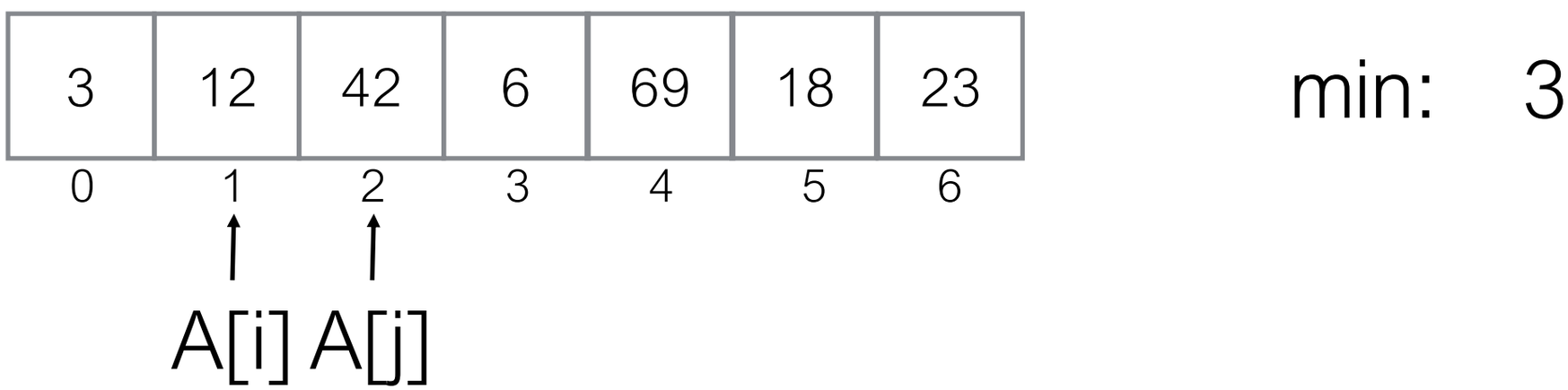
```
function SelSort(A[·], n)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
        swap A[i] and A[min]
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Selection Sort

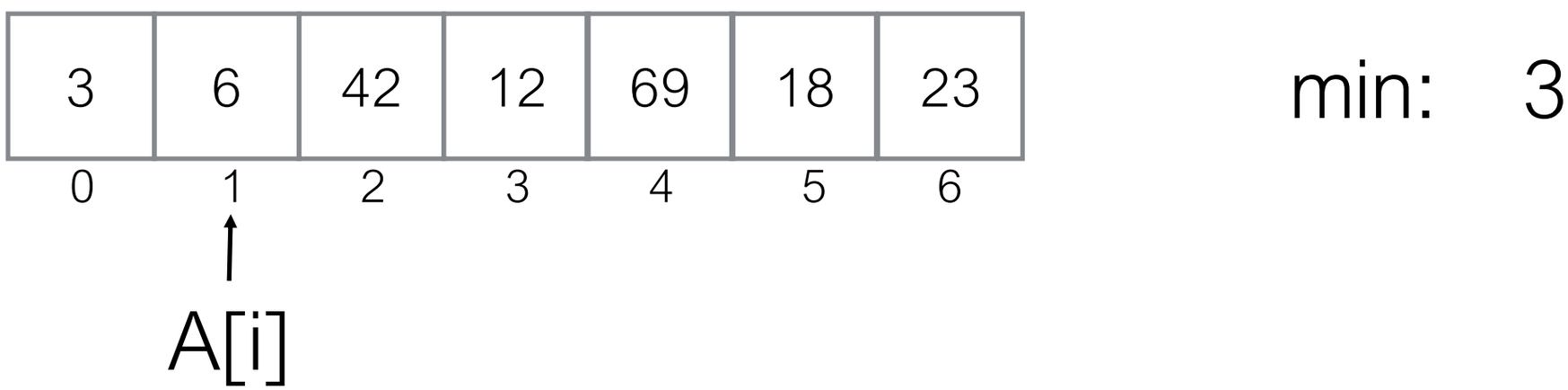
```
function SelSort(A[·], n)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
        swap A[i] and A[min]
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Selection Sort

```
function SelSort(A[·], n)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
        swap A[i] and A[min]
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Selection Sort

```
function SelSort(A[·], n)
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min] then
                min ← j
        swap A[i] and A[min]
```

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

Input size *n*: length of the array

Basic operation: comparison A[j] < A[min]

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Analysis

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i)$$

Using $\sum_{i=l}^{u} 1 = \underbrace{1+1+\cdots+1}_{u-l+1 \text{ times}} = u-l+1$

$$\sum_{j=i+1}^{n-1} 1 = (n-1-(i+1)+1) = (n-1-i)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Analysis

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

Using $\sum_{i=l}^{u} 1 = \underbrace{1+1+\cdots+1}_{u-l+1 \text{ times}} = u-l+1$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Analysis

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

Using $\sum_{i=l}^{u} 1 = \underbrace{1+1+\cdots+1}_{u-l+1 \text{ times}} = u-l+1$

$$\sum_{i=0}^{n-2} (n-1) = (n-1) \sum_{i=0}^{n-2} 1 = (n-1)(n-2-0+1) = (n-1)^2$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Analysis

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2$$

$$\sum_{i=0}^{n-2} i = \sum_{i=1}^{n-2} i = \frac{(n-2)(n-2+1)}{2} = \frac{(n-2)(n-1)}{2}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Selection Sort Analysis

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

$$\sum_{i=0}^{n-2} (n-1) = (n-1) \sum_{i=0}^{n-2} 1 = (n-1)(n-2-0+1) = (n-1)^2$$

$$\sum_{i=0}^{n-2} i = \sum_{i=1}^{n-2} i = \frac{(n-2)(n-2+1)}{2} = \frac{(n-2)(n-1)}{2}$$

$$C(n) = (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{n(n-1)}{2} \in \Theta(n^2)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Matrix Multiplication

THE UNIVERSITY OF MELBOURNE

function MATRIXMULT($A[\cdot,\cdot], B[\cdot,\cdot], n$) ▷ For $n \times n$ matrices
  for $i \leftarrow 0$ to $n-1$ **do**
    for $j \leftarrow 0$ to $n-1$ **do**
      $C[i,j] \leftarrow 0.0$
      for $k \leftarrow 0$ to $n-1$ **do**
        $C[i,j] \leftarrow C[i,j] + A[i,k] \cdot B[k,j]$
  return C

i: 0

j: 0

k: 1

$$A = \begin{bmatrix} 5 & 7 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 8 & 2 \\ 9 & 6 \end{bmatrix} \quad C = \begin{bmatrix} 103 & \\ & \end{bmatrix}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License
next

Example: Matrix Multiplication

```
function MatrixMult(A[·,·], B[·,·], n)        ▷ For n × n matrices
    for i ← 0 to n − 1 do
        for j ← 0 to n − 1 do
            C[i, j] ← 0.0
            for k ← 0 to n − 1 do
                C[i, j] ← C[i, j] + A[i, k] · B[k, j]
    return C
```

i: 0

j: 1

k: 1

$$A = \begin{bmatrix} 5 & 7 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 8 & 2 \\ 9 & 6 \end{bmatrix} \quad C = \begin{bmatrix} 103 & 52 \\ & \end{bmatrix}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Matrix Multiplication

THE UNIVERSITY OF MELBOURNE

function MatrixMult($A[\cdot,\cdot], B[\cdot,\cdot], n$) ▷ For $n \times n$ matrices
  for $i \leftarrow 0$ to $n-1$ **do**
    for $j \leftarrow 0$ to $n-1$ **do**
      $C[i,j] \leftarrow 0.0$
      for $k \leftarrow 0$ to $n-1$ **do**
        $C[i,j] \leftarrow C[i,j] + A[i,k] \cdot B[k,j]$
  return C

Basic operation: multiplication A[i,k] * B[k,j]

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example: Matrix Multiplication

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

$$\sum_{i=l}^{u} 1 = \underbrace{1 + 1 + \cdots + 1}_{u-l+1 \text{ times}} = u - l + 1$$

$$= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} ((n-1) - 0 + 1) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (n \cdot 1)$$

$$= \sum_{i=0}^{n-1} \left(n \cdot \sum_{j=0}^{n-1} 1 \right) = \sum_{i=0}^{n-1} n^2$$

$$\sum_{i=l}^{u} ca_i = c \sum_{i=l}^{u} a_i$$

$$= n^3$$

$$\in \Theta(n^3)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Analysing Recursive Algorithms

function $\mathrm{F}(n)$
    if $n = 0$ **then return** 1
    else return $\mathrm{F}(n-1) \cdot n$

$$
\begin{aligned}
F(5) &= F(4) \cdot 5 \\
&= (F(3) \cdot 4) \cdot 5 \\
&= ((F(2) \cdot 3) \cdot 4) \cdot 5 \\
&= (((F(1) \cdot 2) \cdot 3) \cdot 4) \cdot 5 \\
&= ((((F(0) \cdot 1) \cdot 2) \cdot 3) \cdot 4) \cdot 5 \\
&= ((((1 \cdot 1) \cdot 2) \cdot 3) \cdot 4) \cdot 5 \\
&= 5!
\end{aligned}
$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Analysing Recursive Algorithms

THE UNIVERSITY OF MELBOURNE

function $\mathrm{F}(n)$
    if $n = 0$ **then return** 1
    else return $\mathrm{F}(n-1) \cdot n$

Basic operation: multiplication

We express the cost **recursively** (as a **recurrence relation**)

$$M(0) = 0$$
$$M(n) = M(n-1) + 1$$

Need to express M(n) in **closed form** (i.e. non-recursively)

Try: **“telescoping”** aka **“backward substitution”**

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Telescoping
(aka Backward Substitution)

$$M(n) = M(n-1) + 1 \quad M(0) = 0$$

What is M(n-1) ?

$$M(n-1) = M((n-1)-1) + 1$$
$$= M(n-2) + 1$$

$$M(n) = (M(n-2) + 1) + 1$$
$$= M(n-2) + 2$$
$$= (M(n-3) + 1) + 2$$
$$= M(n-3) + 3$$

$$\vdots$$
$$= M(n-n) + n$$
$$= M(0) + n$$
$$= n$$

Closed form:

$$M(n) = n$$

Complexity:

$$M(n) \in \Theta(n)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

```
function BinSearch(A[·], lo, hi, key)
    if lo > hi then return −1
    mid ← lo + (hi − lo)/2
    if A[mid] = key then return mid
    else
        if A[mid] > key then
            return BinSearch(A, lo, mid − 1, key)
    else return BinSearch(A, mid + 1, hi, key)
```

lo: 0

hi: 6

key: 41

mid: 3

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

BinSearch(A,0,6,41)

BinSearch(A,4,6,41)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

THE UNIVERSITY OF MELBOURNE

```
function BinSearch(A[·], lo, hi, key)
    if lo > hi then return −1
    mid ← lo + (hi − lo)/2
    if A[mid] = key then return mid
    else
        if A[mid] > key then
            return BinSearch(A, lo, mid − 1, key)
    else return BinSearch(A, mid + 1, hi, key)
```

lo: 4

hi: 6

key: 41

mid: 5

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

BinSearch(A,4,6,41)

BinSearch(A,4,4,41)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

```
function BinSearch(A[·], lo, hi, key)
    if lo > hi then return −1
    mid ← lo + (hi − lo)/2
    if A[mid] = key then return mid
    else
        if A[mid] > key then
            return BinSearch(A, lo, mid − 1, key)
    else return BinSearch(A, mid + 1, hi, key)
```

lo: 4

hi: 4

key: 41

mid: 4

A:

4	9	13	22	41	83	96
0	1	2	3	4	5	6

BinSearch(A,4,4,41)

returns 4

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Example:
Binary Search in Sorted Array

THE UNIVERSITY OF MELBOURNE

```
function BinSearch(A[·], lo, hi, key)
    if lo > hi then return −1
    mid ← lo + (hi − lo)/2
    if A[mid] = key then return mid
    else
        if A[mid] > key then
            return BinSearch(A, lo, mid − 1, key)
    else return BinSearch(A, mid + 1, hi, key)
```

Basic operation: key comparison A[mid] = key

$C(0) = 0$ $\qquad\qquad$ $C(n) = C(n/2) + 1$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Telescoping

A **smoothness rule** allows us to assume that n is a power of 2

$$C(0) = 0 \qquad C(n) = C(n/2) + 1$$

$$\begin{aligned} C(n) &= C(n/2) + 1 \\ &= (C(n/4) + 1) + 1 \\ &= ((C(n/8) + 1) + 1) + 1 \\ &= C(n/n) + \log_2 n \\ &= (C(0) + 1) + \log_2 n \\ &= 1 + \log_2 n \end{aligned}$$

$$C(n) \in \Theta(\log n)$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Logarithmic Functions Have Same Rate of Growth

In *O*, *Ω*, *Θ*, expressions we can just write "log" for any logarithmic function no matter what the base is

Asymptotically, all logarithmic behaviour is the same, since

$$\log_a x = (\log_a b)(\log_b x)$$

$$\lim_{n\to\infty} \frac{log_a n}{log_b n} = \lim_{n\to\infty} \frac{(\log_a b)(\log_b n)}{\log_b n} = (\log_a b) \lim_{n\to\infty} 1 = \log_a b$$

So, e.g.: $\Theta(log_a n) = \Theta(log_b n)$

Also, since $\log n^c = c \cdot \log n$ $\log n^c \in \Theta(\log n)$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Summarising Reasoning with Big-Oh

Suppose $t_1(n) \in O(g_1(n))$ $t_2(n) \in O(g_2(n))$

Then the following hold:

$$t_1(n) + t_2(n) \in O(max\{g_1(n), g_2(n)\})$$

(we can throw away smaller summands)

$$c \cdot t_1(n) \in O(g_1(n))$$

(constants can be thrown away too)

$$t_1(n) \cdot t_2(n) \in O(g_1(n) \cdot g_2(n))$$

(for nested loops: count number of times outer loop is executed, multiply by cost of inner loop)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Some Useful Formulas

From Stirling's formula:

$$n! \in O(n^{n+\frac{1}{2}})$$

this is not the time complexity of an algorithm to compute n-factorial

$$\sum_{i=0}^{n} i^2 = \frac{n}{3}(n+\frac{1}{2})(n+1)$$

$$\sum_{i=0}^{n}(2i+1) = (n+1)^2$$

$$\sum_{i=1}^{n} 1/i = O(\log n)$$

See also Cormen's Appendix A or Levitin's Appendix A.

Levitin's Appendix B is a tutorial on recurrence relations.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Road Ahead

- You'll get much more familiar with asymptotic analysis as we use it on algorithms we meet in this course.

- Next week we begin our study of algorithms by looking at **brute force** approaches

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Reminders

- Weekly quizzes
- Diagnostic quizzes
- Plenty of ways to get help: consultation hours, CIS Student Hub, Ed, see Canvas for details.
- The textbook is available in the library catalogue.
- Attempt tutorial questions before tutorials.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Back to Euclid

```
function EUCLID(m, n)
    while n ≠ 0 do
        r ← m mod n
        m ← n
        n ← r
    return m
```

```
function EUCLID(m, n)
    if n = 0 then
        return m
    return EUCLID(n, m mod n)
```

Running time is **linear** in size (in bits) of input, i.e.

$$O(\log(m+n))$$

since the value of m (and n) is at least halved in every two iterations.

Why? After two iterations, m becomes m mod n; also

$$1 < n < m \Rightarrow m \bmod n < m/2$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License